```
# ========================
# INSTALL + IMPORTS
# ========================
!pip install torch torchvision torchaudio -q

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset

# ========================
# SIMPLE DATA (Demo instead of full Cornell dataset)
# ========================
pairs = [
    ("hello", "hi how are you"),
    ("how are you", "i am fine"),
    ("what is your name", "i am a chatbot"),
    ("bye", "goodbye")
]

# ========================
# TOKENIZATION
# ========================
def tokenize(sentence):
    return sentence.lower().split()

vocab = {"<pad>":0, "<sos>":1, "<eos>":2}
for inp, out in pairs:
    for word in tokenize(inp) + tokenize(out):
        if word not in vocab:
            vocab[word] = len(vocab)

inv_vocab = {v:k for k,v in vocab.items()}

def encode(sentence):
    return [vocab[w] for w in tokenize(sentence)]

# ========================
# DATASET
# ========================
class ChatDataset(Dataset):
    def __init__(self, pairs):
        self.data = pairs

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        inp, out = self.data[idx]
        inp_ids = torch.tensor(encode(inp))
        out_ids = torch.tensor([1] + encode(out) + [2])  # <sos>, <eos>
        return inp_ids, out_ids

def collate_fn(batch):
    inputs, outputs = zip(*batch)
    inputs = nn.utils.rnn.pad_sequence(inputs, batch_first=True)
    outputs = nn.utils.rnn.pad_sequence(outputs, batch_first=True)
    return inputs, outputs

loader = DataLoader(ChatDataset(pairs), batch_size=2, collate_fn=collate_fn)

# ========================
# MODEL WITH ATTENTION
# ========================
class Attention(nn.Module):
    def __init__(self, hidden_dim):
        super().__init__()
        self.attn = nn.Linear(hidden_dim*2, hidden_dim)
        self.v = nn.Linear(hidden_dim, 1, bias=False)

    def forward(self, hidden, encoder_outputs):
        seq_len = encoder_outputs.shape[1]
        hidden = hidden.unsqueeze(1).repeat(1, seq_len, 1)
        energy = torch.tanh(self.attn(torch.cat((hidden, encoder_outputs), dim=2)))
        attention = self.v(energy).squeeze(2)
```

```
        return torch.softmax(attention, dim=1)

class Seq2Seq(nn.Module):
    def __init__(self, vocab_size, embed_dim=64, hidden_dim=128):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.encoder = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
        self.decoder = nn.LSTM(embed_dim + hidden_dim, hidden_dim, batch_first=True)
        self.attention = Attention(hidden_dim)
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, src, trg):
        embedded = self.embedding(src)
        encoder_outputs, (hidden, cell) = self.encoder(embedded)

        outputs = []
        input_token = trg[:,0]

        for t in range(1, trg.shape[1]):
            emb = self.embedding(input_token).unsqueeze(1)

            attn_weights = self.attention(hidden[-1], encoder_outputs)
            context = torch.bmm(attn_weights.unsqueeze(1), encoder_outputs)

            dec_input = torch.cat((emb, context), dim=2)
            output, (hidden, cell) = self.decoder(dec_input, (hidden, cell))

            pred = self.fc(output.squeeze(1))
            outputs.append(pred)

            input_token = trg[:,t]

        return torch.stack(outputs, dim=1)

# ========================
# TRAINING
# ========================
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = Seq2Seq(len(vocab)).to(device)

criterion = nn.CrossEntropyLoss(ignore_index=0)
optimizer = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(200):
    for src, trg in loader:
        src, trg = src.to(device), trg.to(device)

        output = model(src, trg)
        output = output.reshape(-1, len(vocab))
        trg = trg[:,1:].reshape(-1)

        loss = criterion(output, trg)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    if epoch % 50 == 0:
        print(f"Epoch {epoch}, Loss: {loss.item():.4f}")

# ========================
# INFERENCE
# ========================
def generate(sentence):
    model.eval()
    src = torch.tensor([encode(sentence)]).to(device)

    with torch.no_grad():
        embedded = model.embedding(src)
        enc_out, (hidden, cell) = model.encoder(embedded)

        input_token = torch.tensor([1]).to(device)  # <sos>
        result = []

        for _ in range(10):
            emb = model.embedding(input_token).unsqueeze(1)
            attn = model.attention(hidden[-1], enc_out)
            context = torch.bmm(attn.unsqueeze(1), enc_out)
```

```
            context = torch.cat((hidden, cell), dim=2)

            dec_input = torch.cat((emb, context), dim=2)
            out, (hidden, cell) = model.decoder(dec_input, (hidden, cell))

            pred = model.fc(out.squeeze(1))
            token = pred.argmax(1).item()

            if token == 2:
                break

            result.append(inv_vocab[token])
            input_token = torch.tensor([token]).to(device)

    return " ".join(result)

# ========================
# TEST
# ========================
print(generate("hello"))
print(generate("how are you"))
```

```
Epoch 0, Loss: 2.8992
Epoch 50, Loss: 0.0195
Epoch 100, Loss: 0.0065
Epoch 150, Loss: 0.0034
hi how are you
i am fine
```